



PDF Download
3677052.3698691.pdf
06 February 2026
Total Citations: 2
Total Downloads: 1329

Latest updates: <https://dl.acm.org/doi/10.1145/3677052.3698691>

RESEARCH-ARTICLE

Mixtures of Experts for Scaling up Neural Networks in Order Execution

KANG LI, University of Oxford, Oxford, Oxfordshire, U.K.

MIHAI CUCURINGU, Oxford-Man Institute of Quantitative Finance, Oxford, Oxfordshire, U.K.

LEANDRO SÁNCHEZ-BETANCOURT, Oxford-Man Institute of Quantitative Finance, Oxford, Oxfordshire, U.K.

TIMON WILLI, University of Oxford, Oxford, Oxfordshire, U.K.

Open Access Support provided by:

Oxford-Man Institute of Quantitative Finance

University of Oxford

Published: 14 November 2024

[Citation in BibTeX format](#)

ICAIF '24: 5th ACM International
Conference on AI in Finance
November 14 - 17, 2024
NY, Brooklyn, USA

Mixtures of Experts for Scaling up Neural Networks in Order Execution

Kang Li*

Department of Statistics, University of Oxford
United Kindom
kang.li@stats.ox.ac.uk

Mihai Cucuringu

Department of Statistics & Mathematical Institute and
Oxford-Man Institute of Quantitative Finance, University
of Oxford
United Kingdom
mihai.cucuringu@stats.ox.ac.uk

Leandro Sánchez-Betancourt

Mathematical Institute, Oxford-Man Institute of
Quantitative Finance, University of Oxford
United Kingdom
sanchezbetan@maths.ox.ac.uk

Timon Willi

Foerster Lab for AI Research, University of Oxford
United Kingdom
timon.willi@eng.ox.ac.uk

Abstract

We develop a methodology that employs mixture of experts to scale up the parameters of reinforcement learning (RL) models in optimal execution tasks. The innovation of our approach stems from a spectral clustering-driven methodology for customized training of RL agents. First, we cluster the data using domain-specific knowledge, creating distinct subsets for specialized learning. Second, for each cluster, we train individual customized RL experts. Third, we train a router layer which serves as a crucial branching mechanism for dynamically assigning the most appropriate expert for each step during an episode. We deploy and test this methodology on real data of the Amazon stock on NASDAQ, and provide evidence that our approach improves the profitability of execution tasks by approximately one basis point when compared to a single expert scenario.

Keywords

Reinforcement Learning; Mixture of Experts; Optimal Execution; Limit Order Books.

ACM Reference Format:

Kang Li, Mihai Cucuringu, Leandro Sánchez-Betancourt, and Timon Willi. 2024. Mixtures of Experts for Scaling up Neural Networks in Order Execution. In *5th ACM International Conference on AI in Finance (ICAIF '24)*, November 14–17, 2024, Brooklyn, NY, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3677052.3698691>

1 Introduction

Large-scale training by end-to-end neural networks has been a focus of attention in optimal execution for trading applications. Recent advancements, such as those presented in JAXLOB [20] and PUREJAXRL

[31], have paved the way for large-scale reinforcement learning (RL) training in order execution. These innovations have significantly accelerated training environments and RL algorithms through GPU acceleration, addressing key computational challenges in the field.

With these computational hurdles largely overcome, the path is now clear to advance the algorithms that solve order execution tasks across a large number of scenarios. However, a critical challenge remains: the need to scale up neural networks to obtain greater representational capacity. This is becoming increasingly crucial if one is to capture the intricate dynamics and vast state spaces encountered in real-world trading scenarios.

Advancements in supervised learning have demonstrated a clear correlation between model size and performance, summarized in the following scaling law: a model's performance scales proportionally to its size. However, in the domain of RL, this relationship appears to be more complex. Contrary to expectations, increasing the parameters of an RL model often leads to a detriment in its performance [38].

Recently, Obando-Ceron et al. [38] and Willi et al. [51] proposed a mixture of experts (MoE) approach to RL tasks, successfully scaling parameter count when applied to Atari games. We aim to implement a similar methodology in the context of order execution. Unlike Atari in Obando-Ceron et al. [38], order execution data exhibits a low signal-to-noise ratio, presenting challenges for model training. Consequently, it is challenging to train the MoE module from scratch following the approach in [38]. Instead, we adopt the BRANCHTRAINMIX (BTX) method proposed by [47], combining multiple domain-specialized models into a single MoE model. In this approach, each expert is pre-trained on specific scenarios and subsequently a router is trained to select the experts. For instance, in an MoE setting with two experts, one expert could be specifically pre-trained for high-volatility scenarios, while the other expert could be responsible for managing low-volatility scenarios.

To determine the appropriate scenarios for each expert, we employ a spectral clustering approach based on the properties of the data in each scenario. This method involves generating statistical summaries or property vectors from the data of each scenario, serving as the basis for clustering. This approach aligns with recent

*K.L. is the first author, the rest of the authors are listed alphabetically.



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

ICAIF '24, November 14–17, 2024, Brooklyn, NY, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1081-0/24/11
<https://doi.org/10.1145/3677052.3698691>

developments in the field that leverage the synergy between clustering and RL. For example, [32] introduced a framework called clustered RL (CRL), which enhances efficient exploration in RL by clustering collected states into groups. Our methodology builds upon these concepts, adapting them to the specific challenges of order execution in financial markets.

Main contributions. We provide an RL framework for scalable order execution in limit order books. The key aspects in our approach are the clustering of the data based on relevant domain knowledge summary statistics, and the dynamic assignment of pre-trained RL experts (optimized within each cluster). We demonstrate the outperformance of our method when compared to single-expert benchmarks using real data from the US equity market.

Paper structure. Section 2 provides a background on the main ingredients of our proposed pipelines, namely limit order books, optimal execution, spectral clustering, and mixtures of experts. Section 3 surveys the relevant MoE literature, emphasizing the main challenges therein. Section 4 details the main components of our proposed RL framework, with an eye toward the incorporation of relevant financial domain knowledge. Section 5 summarizes the experimental setup and shows the numerical results we obtain when comparing against the benchmarks. Finally, Section 6 concludes and puts forth future potential research directions.

2 Background

2.1 Limit Order Books (LOBs)

The limit order book (LOB) encompasses the entirety of active limit orders, along with the algorithmic process for matching incoming orders with existing ones. When an incoming liquidity-taking order “crosses the spread”, the agent submitting the order is willing to accept a price that is less favorable than those sitting in the LOB on the other side, and thus the matching process begins. This process pairs the incoming order with existing orders in the book according to predetermined rules, such as price-time priority. The successful pairing of compatible orders results in the execution of a trade, finalizing the transaction between the matched parties. The state of the LOB can be accurately reproduced by utilizing the initial state of the book together with the sequence of incoming orders.

Limit order book simulators. There are a number of implementations of LOB simulators. Some of them use agent-based models, while others utilize historical data replay. Here, we employ the later approach. One notable attempt in this field is the development of numpy-based simulators [26]. Building upon the same concept, JAX-LOB emerged as a Jax-based limit order book simulator [20], which operates on the GPU, enabling large-scale parallelization. This advancement is particularly significant for RL applications because Jax-accelerated environments enhance the training performance of RL policies [12, 13, 33, 50].

2.2 Order execution

We briefly discuss the predominant methodologies currently employed in order execution. Optimal order execution acquires or liquidates a predetermined quantity of securities within a designated period, with the objective of achieving the most favorable average price while maintaining a low-risk profile. This process

typically focuses on the reduction of costs associated with market impact.

Order execution with optimal control. As a stochastic control problem, “optimal trade execution” is a well-studied problem within the mathematical finance community. Most models prescribe how the market digests the order flow, translating it into price impact. Then, the objective is to optimize a given goal functional (usually profit-and-losses adjusted by inventory control) and characterize optimality either using Hamilton-Jacobi-Bellman equations or variational analysis tools; some examples of the ideas being pursued are [1, 3, 8, 10, 15, 19, 24, 36]. We refer the reader to [6, 23] for a textbook treatment of these control problems and [14] for a recent review article.

In the present article, we employ RL for the optimal execution task. Thus, there is no need to prescribe the dynamics of the system, nor to state the precise way in which actions impact the future evolution of the state (price impact). However, our goal still remains aligned with that of previous lines of work, i.e., we seek to minimize execution costs and maximize the profit gains received from the execution program.

Order execution with deep RL. The evolution of order execution methodologies has seen significant advancements in recent years. Ning et al. [37] introduced a variation of the double deep Q-learning algorithm, using experience replays and encoding transaction costs and price impact with a quadratic penalty on trade sizes. Similarly, [7] used double deep Q-learning and reinforced deep Markov models to study optimal execution within a triplet of currencies. In both works, the price impact of the orders from other RL agents and their reactions is not directly studied.¹ To overcome this, [34] employed the ABIDES [4] simulator, allowing for more dynamic and reactive market environments. Concurrently, [17] went beyond utilizing a simulator with historical data and also expanded the scope to learn the placement of up to three levels of the LOB simultaneously, while additionally exploring a set of market features. Zhang et al. [53] approached order execution from an offline RL perspective, addressing the critical issues of overfitting and out-of-sample effectiveness, with the help of learning compact representations for context.²

2.3 Spectral Clustering and applications to finance

Spectral clustering methods have become essential tools with diverse applications in network analysis and machine learning [49]. Their appeal is twofold: they are computationally scalable thanks to advanced eigensolvers, and they can be theoretically analyzed using well-defined stochastic block models, which assess their robustness to noise and the sparsity of the measurement graph. Additionally, understanding the spectrum of the adjacency matrix and its Laplacians is vital for developing efficient algorithms with performance guarantees, presenting a rich array of mathematical challenges.

¹Arguably, the permanent price impact and the randomness in their models should capture aspects of these reactions, but it is not explicitly addressed as it would be in an agent-based simulator of the market.

²The work in [35] falls within this category for the class of propagator models for the price impact. The authors study an offline learning problem where the agent estimates the unknown price impact kernel from a given dataset, and then designs strategies to liquidate the risky asset.

There is a growing body of literature employing clustering techniques in financial applications. For example, León et al. [30], Tola et al. [48] first clustered the asset correlation matrix to extract groups of stocks that exhibit similar co-movement patterns, with the end goal of constructing Markowitz portfolios within each cluster. [5] evaluated various clustering algorithms for financial datasets, including time series, and discuss the pros and cons of each method. [41] applied clustering techniques to detect and identify correlation regimes, aiding in financial risk management and portfolio construction. Recently, [27] proposed a framework employing graph clustering algorithms to identify groups of correlated stocks that co-move, followed by the construction of mean-reverting arbitrage portfolios within each cluster; their work demonstrated that the clustering facilitated the development of statistical arbitrage strategies delivering economically significant profits.

2.4 Mixture of Experts (MoE)

Mixture of experts is a key concept in machine learning that builds on the divide-and-conquer principle. These methods employ a set of $N \in \mathbb{N}$ experts sub-networks, which are activated by the gating network, routing the incoming tokens to $k \leq N$ experts [45]. The system activates fewer experts than the total available, with k being the number chosen, and in this work we take $k = 1$, thereby inducing sparser activations [18]. Their strong empirical results of MoEs has sparked significant research interest in recent years [18, 28, 29, 42, 45, 52, 54].

Soft MoE. Hard assignments of inputs to experts introduce a number of challenges such as unstable training and failure in scaling the number of experts [18, 43]. To address some of these challenges, soft MoE is introduced by [43], which is a fully differentiable soft assignment of inputs-to-experts, replacing hard token assignments relying solely on routers. The soft assignment involves calculating learned combinations of token-specific weightings for each expert, and then taking the average of their outputs. This approach replaces the traditional hard assignment method.

3 Related Works

3.1 MoE in RL

Recent research [51] has shown that naïvely scaling up network parameters does not necessarily result in improved performance. However, [39] demonstrates that the mixture of experts (MoE) mechanism can simultaneously increase network parameter size and enhance model performance. In this paper, we empirically demonstrate the benefits of MoEs on the performance of value-based agents across a diverse set of training regimes.

Sparsity in MoE improves performance. To elaborate on how MoE enhances network performance, [39, 51] show that MoEs induce structured sparsity in a network, a characteristic that has been demonstrated to improve network performance in previous studies [16, 22]. [39] and [51] suggest that there is still much room for exploration and understanding of the role sparsity can play in training deep RL networks, especially for parameter scalability. This insight serves as inspiration for our research. In our study, we extend the investigation beyond DQN to further explore RNN and PPO architectures.

Soft MoE helps parameter scalability. Building upon the insights on sparsity, [39] suggests that soft MoEs can effectively leverage and exploit parameter scalability. This conclusion is drawn by comparing the performance of soft MoEs with scaled base architectures and MoEs with reduced expert dimensionality, where soft MoE maintained its performance even with fewer experts. Additionally, [39] demonstrate that incorporating soft MoEs [43] strongly improves the performance of various deep RL agents, and performance improvements scale with the number of experts used [40].

3.2 MoE with pre-trained experts.

The Branch-Train-MiX (BTX) method in [47] combines multiple domain-specialized models into a single MoE model. The framework initializes MoEs from separately trained domain models instead of random initialization. BTX first (i) branches multiple copies from a seed base model, then (ii) trains these copies in parallel on different domain-specific datasets to create expert models, and (iii) mixes these experts by combining their feedforward layers into a unified MoE model, followed by (iv) fine-tuning to learning of the token-level routing, ultimately producing an MoE model with enhanced capabilities across multiple domains. This research is related to our work because we also use pre-trained models as experts in the MoE.

This innovative approach offers several notable advantages. For example, it allows for the efficient transfer of domain-specific knowledge into a unified MoE architecture. Another key benefit of this pre-trained MoE approach over traditional MoE models lies in its ability to parallelize and asynchronously execute expert training. This significantly reduces communication overhead and enhances training efficiency, resulting in improved overall throughput. Lastly, by using pre-trained models, BTX achieves better performance than training MoEs from scratch. Empirical results demonstrate that BTX consistently outperforms the benchmarks in [47] across all tasks, underscoring the efficacy of its learned routing mechanism through MoE fine-tuning.

4 Model

4.1 RL task setting

This section introduces the basic setting for the RL order execution task, including the simulator environment and the RL agent.

Simulator environments and the data. In this study, we employed the GPU-accelerated LOB simulator developed by [20]. Under this setting, historical order replay was utilized. Our step definition encompasses one order from the RL agent at the step initiation, followed by 100 orders from other market participants. The episode structure works as follows. For a given day of data, between 9:30 am and 3:00 pm, we pass through the data in 100-order steps until the first time a minute of data has been gathered. Thus, the number of steps per episode is random and can be thought as the number of times that one hundred orders were collected to complete one minute of data. Note that we work on the number-of-orders clock as opposed to the time clock. Our order submission strategy employs a two-phase approach within each episode. For the majority of steps within an episode, we submit limit orders. To ensure the task completion and the execution of all orders, we transition to market orders at a predetermined step. This is to guarantee that towards

the end of the episode, we have liquidated the position. Specifically, market orders are submitted two steps before the episode horizon; we allow for two steps to submit the remaining market orders to allow for the scenario where there is not enough liquidity to match the order submission. We use LOBSTER [25] orderbook data and message data, incorporating the top 10 price levels for AMZN (Amazon.com, Inc).

Action space. Here, we adopt a continuous action space with two dimensions. The actions correspond to the volumes attached to the passive order and the aggressive order. More precisely, the first dimension is for the passive order and the second dimension is for the aggressive order; the price limit in these orders (P_{passive} and $P_{\text{aggressive}}$ respectively) are defined in (1) and (2):

$$P_{\text{aggressive}} = \begin{cases} P_{\text{ask}} * 1.2 & \text{if task is buy} \\ P_{\text{bid}} * 0.8 & \text{if task is sell} \end{cases}, \quad (1)$$

and

$$P_{\text{passive}} = \begin{cases} P_{\text{bid}} & \text{if task is buy} \\ P_{\text{ask}} & \text{if task is sell} \end{cases}, \quad (2)$$

where P_{bid} and P_{ask} are the best bid price and the best ask price in the LOB at the time of taking the actions.

Reward function. Similar to [21], the RL agent is rewarded for outperforming the volume weight average price of all market participants during each step and outperforming price trends between steps. The reward R of an episode is defined as

$$R = \sum_j Q_j (P_j - P_j^{\text{VWAP}}) + \lambda \sum_j Q_j (P_j^{\text{VWAP}} - P_{\text{init}}) \quad (3)$$

where Q_j is the quantity sold by the RL agent during step j , P_j is price obtained by the RL agent when selling Q_j units, P_{init} is the midprice at the start of one scenario, and

$$P_j^{\text{VWAP}} = \frac{\sum_i Q_{j,i} P_{j,i}}{\sum_i Q_{j,i}}, \quad (4)$$

where $\{Q_{j,i}, P_{j,i}\}_i$ are the quantities and prices of all transactions that happened within the one hundred orders in step j ; if there were to be no transactions, we set $P_j^{\text{VWAP}} = P_{j-1}^{\text{VWAP}}$.

In equation (3), the key idea is to maximize the execution prices P_j that the RL agent receives for the Q_j units.³ The parameter $\lambda \in [0, 1]$ balances the benchmark price against which we compare the execution price P_j . If $\lambda = 0$, the RL agent is rewarded to outperform P_j^{VWAP} , and if $\lambda = 1$, the RL agent is rewarded to outperform P_{init} ; values in the middle correspond to a convex combination of these two scenarios.

State Space and Observation Space We follow the state space definition in [20], which we enhance with intra-step variables and agent-specific variables. Regarding the observation space, we apply a standard normalization procedure with the mean and standard deviation from the historical data. The resulting normalized values are constrained within the range of $[-1, 1]$, facilitating their usage in neural networks. The intra-step and agent-specific variables are as follows:

- Intra-step variables:
 - Best level prices
 - Best level volumes

- Agent-specific variables:

- The episode-initial mid-price
- The total volume for the execution task
- The executed quantity up to the current step
- The average price of the executed order up to the current step

4.2 MoE setting

This section delineates our methodology for incorporating multiple experts within an execution task by leveraging MoE algorithms. Our approach follows a three-stage procedure. We initially cluster the data based on domain-specific knowledge, creating distinct subsets for specialized learning. Then we train individual RL experts for each cluster, allowing for the incorporation of subdomain-specific knowledge. Finally, a router layer is trained, serving as a crucial branching mechanism for dynamically assigning the most appropriate expert for each step during an episode. The process is visualized in Figure 1.

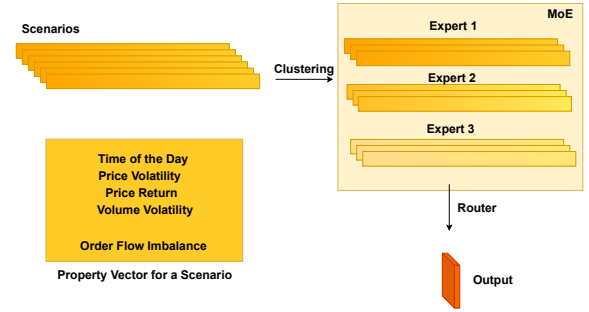


Figure 1: Diagram for MoE.

Domain clustering We define properties for each episode sampled from the dataset. The sampled episode i is represented as a property vector $\text{Property}^i \in \mathcal{P}$ defined in (5).

$$\text{Property}^i = \begin{cases} T^i & \text{time of day} \\ P_{\text{vol}}^i & \text{price volatility} \\ P_{\text{ret}}^i & \text{price return} \\ V_{\text{vol}}^i & \text{volume volatility} \\ V_{\text{sum}}^i & \text{sum of traded volume} \\ \text{OFI}^i & \text{order flow imbalance} \end{cases} \quad (5)$$

where $\mathcal{P}$ is the set of possible vector values.

We use spectral clustering $S : \mathcal{E} \rightarrow \{1, 2, \dots, N\}$ based on the properties of episodes. The episodes dataset $\mathcal{E}$ is then divided into $\mathcal{E}_i = \{e \in \mathcal{E} \mid S(e) = i\}$, for $i \in \{1, 2, \dots, N\}$, where N is the number of clusters and the number of experts in the subsequent MoE router.

By dividing the dataset into several domains, we can attain two benefits: firstly, it allows each expert to learn domain-specific knowledge better; secondly, the determination of domains introduces prior knowledge into the model.⁴

³In the reward R one can easily add running inventory penalties or similar risk control terms.

⁴We also utilized residual policy learning to exploit the prior knowledge, which will be described in detail in Appendix A.1

Pre-trained experts. The pre-trained models in the MoE module are the experts trained under the RL tasks described in Section 4.1, based on the data from the domain clustering. The parameters of these experts are fixed during the training of the MoE router layer.

Router layer and branching. The naive approach is to pre-train multiple experts and then integrate them with a trainable router layer. This layer determines the most appropriate expert for handling the current episode based on progressively observed states, effectively assigning inputs to specific expert networks. Like [47], we maintain the pre-trained experts' parameters as fixed during the training process, focusing solely on optimizing the network parameters of the router to increase the branching capability. The details of the router layer and branching can be found in Figure 2.

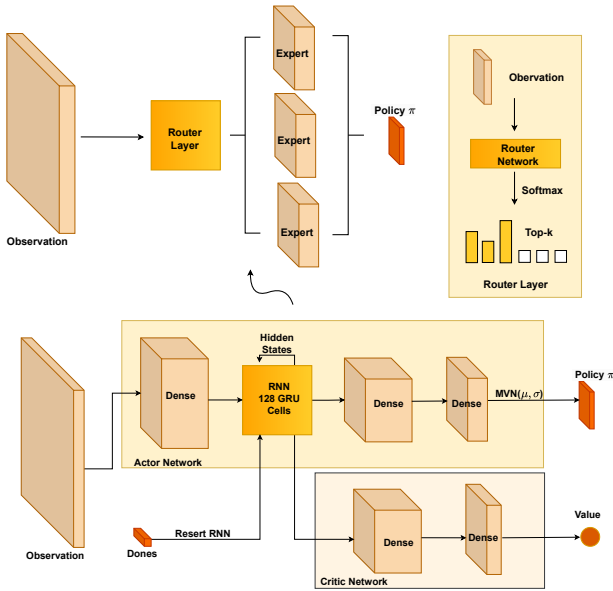


Figure 2: Diagram for the MoE layer and architecture for the expert's networks.

5 Experiments

This section provides the details of the experiments we deploy. The goal of the experiments is to explore the advantage of scaling the parameters of deep RL agents in the order execution task. Here we utilize a recurrent PPO algorithm [44]; in Table 1 we summarize the values of the configuration parameters we employ. The incorporation of recurrent neural networks, specifically gated recurrent units (GRUs) [11], enables the model to maintain a hidden state, thus capturing temporal dependencies in the trading environment. This memory mechanism is crucial for processing sequential information, such as previously executed orders, which informs the decision-making process for subsequent order submissions.

In the environment setup, we adopt a delta action type, where the agent learns deviations from a baseline time-weighted average price (TWAP) strategy. This approach allows the agent to refine

an established trading strategy rather than learning from scratch, potentially accelerating convergence and improving performance. Further details on this implementation can be found in Appendix A.1. We set the reward shaping parameter λ to zero. This configuration focuses the agent's objective on outperforming the volume-weighted average price at each step. By simplifying the reward structure, we enable the agent to learn more effectively in the trading environment. This design choice emphasizes immediate performance optimization while avoiding the additional complexity of long-term price prediction, which can be particularly challenging in highly dynamic markets.

Table 1: PPO configuration parameters

Category	Parameter	Value
Optimization	Learning rate	2.5×10^{-5}
	Entropy coefficient	0.0
	Number of environments	256
	Total timesteps	5×10^7
	Number of minibatches	4
	Update epochs	10
	Number of steps	50
PPO specific	Number of updates	1000
	Clip epsilon	0.2
	Discount factor (γ)	0.999
	GAE lambda	0.99
	Value function coefficient	1.0
Network architecture	RNN type	GRU
	Hidden size	128
	Activation function	ReLU
	Number of experts	{1,2,4,8}
	Top-k	1
Environment setup	Task side	Sell
	Reward lambda	0.0
	Action type	Delta
	Episode time	60 seconds
	Data type	Fixed time

In our experimental design, we align the transition collection process with the episodic structure of the environment. That is, we set the number of steps for collecting transitions large enough so that it often covers one complete episode; recall that the average episode horizon is 10 steps, as shown in Table 2. This offers several advantages. Firstly, it ensures complete trajectory capture, allowing the agent to learn from comprehensive action-reward sequences and facilitating more effective policy updates. Moreover, this method preserves temporal coherence, maintaining the integrity of the temporal structure of the trading process. Consequently, the agent can learn long-term dependencies and develop strategies that span entire trading sessions, a crucial capability in the dynamic and complex realm of algorithmic trading.

Training. We train on a substantial dataset comprising 390 scenarios (one per minute of trading), encompassing approximately 360,000 orders derived from message data, collected on April 1st,

Table 2: Statistical summary of episodic horizons and task volumes.

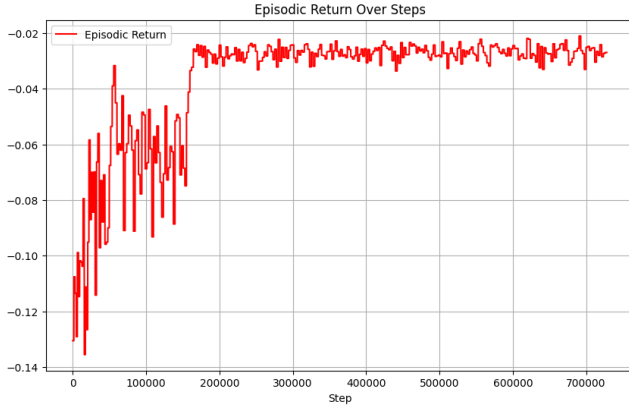
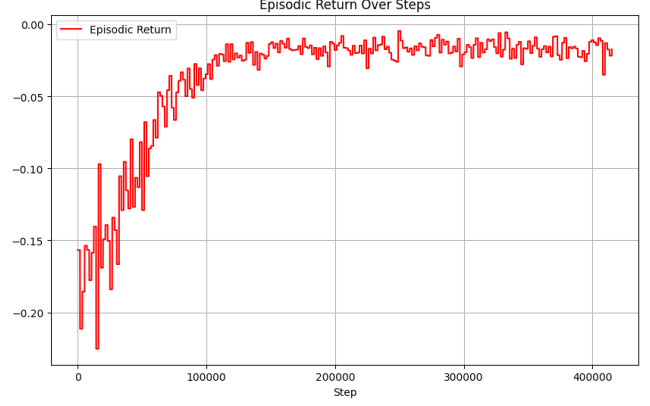
Statistic	Mean	Median	Standard deviation
Episodic horizon	9.66	8.0	6.17
Task volume	59.43	31.0	80.34

2021. Each scenario is configured with a duration of 1 minute. The scenarios incorporate historical data, where (by definition) each step contains 100 orders from other market participants. Consequently, the horizon of each scenario varies; the average time horizon is 10 steps.

To provide a comprehensive overview of our experimental setup, Table 2 shows the key characteristics of the training scenarios used to train the reinforcement learning agent in the same environment. The volumes we seek to execute are 5% of the trading volume within each scenario. This configuration represents a significant liquidity-taking role in execution, where the execution price achieved by the RL agent at each step is highly likely to be less favorable than the volume-weighted average price (VWAP) for that step; because the VWAP incorporates both buy and sell transactions from market participants.

Given the challenging nature of this liquidity-taking task, it is not surprising to observe initially negative returns, as shown in Figure 3 and Figure 4. However, the upward trajectory of returns during training, culminating in convergence, indicates that the agents have successfully learned their policies.

The training curves depicted in Figure 3 and Figure 4 reveal more insights into the performance of both the mixture of experts (MoE) and single expert (SE) models. While both approaches exhibit stable training progression, the SE model, although demonstrating consistent improvement, achieves slightly lower returns compared to its MoE counterpart. This difference suggests that the MoE architecture may offer enhanced adaptability to the intricacies of the LOB environment. To provide a clearer visualization of the training progression, the curves incorporate a moving average of ten data points, effectively smoothing out short-term fluctuations and highlighting the overall trends in agent performance.

**Figure 3: Training curve of MoE with 8 trained experts.****Figure 4: Training curve of SE with equivalent parameters to the MoE with 8 experts.**

Next, we evaluate the robustness and generalization capabilities of our trained models, for which we conduct a comprehensive assessment using a diverse set of 390 previously unseen scenarios. These scenarios were derived from data collected on April 5, 2021. For each scenario, we performed a rollout and calculated the final average price achieved by our models. To enable meaningful comparison across different models and scenarios, particularly given the significant price variations that can occur within a single trading day, we developed a standardized metric. This approach allows for fair evaluation despite the potential for substantial differences in average prices across scenarios. We introduce the concept of relative “outperformance”, which quantifies the relative improvement in average price performance. This metric is expressed in basis points (bp), providing a normalized measure of model superiority. We present the outperformance of our models over the benchmark as follows. At test date i , the outperformance is denoted as Outperformance^i

$$\text{TotalRevenue}_m^i = \sum_t \text{ExecutedQuantity}_m^{i,t} * \text{ExecutedPrice}_m^{i,t},$$

$$\text{Outperformance}^i = \frac{\text{TotalRevenue}_{\text{MoE}}^i - \text{TotalRevenue}_{\text{SE}}^i}{\text{TotalRevenue}_{\text{SE}}^i}, \quad (6)$$

where $m \in \{\text{MoE}, \text{SE}\}$ is the model, t is the step-index in a day, and TotalRevenue_m is the total revenue of the execution task by model m .⁵

Table 3 presents a comparative analysis of the out-of-sample performance between the MoE and SE models. As we increase the number of experts, a pattern emerges: the MoE model begins to outperform its SE counterpart, demonstrating an average advantage of approximately 1 basis point. Notably, this performance gap widens as we further increase the number of experts from 2 to 4.

These results provide compelling evidence for the efficacy of the MoE mechanism in enhancing the parametric capacity of the RL agent. The observed performance improvements suggest that

⁵In this paper we focus on selling the stock but it can be generalized to similar scenarios, such as a buying task.

the MoE approach effectively leverages its expanded parameter space, leading to more nuanced and effective trading strategies. This finding is particularly significant as it points to the potential for scaling up RL agents in trading.

Table 3: Out-of-sample outperformance: mixtures of experts versus averaged single experts and equivalent-parameter single expert. Averaged single experts represents the average performance of multiple independently trained single experts on various datasets. Equivalent-parameter single expert refers to a single expert model with parameters equivalent to the corresponding MoE, achieved by increasing the hidden size of the GRU and the width of the Dense layer.

Outperformance over	Num. of experts		
	2	4	8
Equivalent-parameter SE	0.27	0.29	0.09
Averaged SE	0.98	1.21	0.92

6 Conclusions

We showed how to employ mixtures of experts for scaling up neural networks in order execution. Indeed, the RL agents with MoEs consistently outperform their SE counterparts. These results provide evidence that the MoE mechanism enhances the performance of RL agents, showing the potential for scaling up RL algorithms in trading.

In terms of potential future directions, it would be interesting to study experts on two stocks that exhibit an established lead-lag relationship [2, 9] and use the information from the leader stock to help execution on the lagging stock. Within this setting, the state of the agent will be a combination of the state from the leading stock and the state from the lagging state. It is also worth exploring to train jointly on multiple assets to account for cross-asset interactions and cross-impact. Another future direction pertains to extending our study to larger baskets of assets to showcase generalizability. Here, we could employ another layer of clustering at the asset level for the purpose of pooling together assets that exhibit similar microstructure (or fundamental) characteristics.

References

- [1] Robert Almgren and Neil Chriss. 2001. Optimal execution of portfolio transactions. *Journal of Risk* 3 (2001), 5–40.
- [2] Stefanos Bennett, Mihai Cucuringu, and Gesine Reinert. 2022. Lead-lag detection and network clustering for multivariate time series with an application to the US equity market. *Machine Learning* 111, 12 (Dec. 2022), 4497–4538. <https://doi.org/10.1007/s10994-022-06250-4>
- [3] Philippe Bergault, Fayçal Drissi, and Olivier Guéant. 2022. Multi-asset Optimal Execution and Statistical Arbitrage Strategies under Ornstein–Uhlenbeck Dynamics. *SIAM Journal on Financial Mathematics* 13, 1 (2022), 353–390.
- [4] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. 2019. ABIDES: Towards High-Fidelity Market Simulation for AI Research. <https://doi.org/10.48550/arXiv.1904.12066> arXiv:1904.12066 [cs].
- [5] Fan Cai, Nhien-An Le-Khac, and Tahar Kechadi. 2016. Clustering approaches for financial data analysis: a survey. *arXiv preprint arXiv:1609.08520* (2016).
- [6] Álvaro Cartea, Sebastian Jaimungal, and José Penalva. 2015. *Algorithmic and high-frequency trading*. Cambridge University Press.
- [7] Álvaro Cartea, Sebastian Jaimungal, and Leandro Sánchez-Betancourt. 2023. Reinforcement Learning for Algorithmic Trading. In *Machine Learning and Data Sciences for Financial Markets: A Guide to Contemporary Practices*, Charles-Albert Lehalle and Agostino Capponi (Eds.). Cambridge University Press.
- [8] Álvaro Cartea, Imanol Pérez Arribas, and Leandro Sánchez-Betancourt. 2022. Double-Execution Strategies Using Path Signatures. *SIAM Journal on Financial Mathematics* (Nov. 2022). <https://doi.org/10.1137/21M1456467> Publisher: Society for Industrial and Applied Mathematics.
- [9] Álvaro Cartea, Mihai Cucuringu, and Qi Jin. 2023. Detecting Lead-Lag Relationships in Stock Returns and Portfolio Strategies. *SSRN 4599565* (2023). <https://ssrn.com/abstract=4599565>
- [10] Álvaro Cartea and Leandro Sánchez-Betancourt. 2023. Optimal execution with stochastic delay. *Finance and Stochastics* 27, 1 (Jan. 2023), 1–47. <https://doi.org/10.1007/s00780-022-00491-w>
- [11] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. 2014. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. <https://doi.org/10.48550/arXiv.1412.3555> arXiv:1412.3555.
- [12] Samuel Coward, Michael Beukman, and Jakob Foerster. 2024. JaxUED: A simple and useable UED library in Jax. <http://arxiv.org/abs/2403.13091> arXiv:2403.13091 [cs].
- [13] Christian Schroeder de Witt, Tarun Gupta, Denys Makoviichuk, Viktor Makoviy-chuk, Philip H. S. Torr, Mingfei Sun, and Shimon Whiteson. 2020. Is Independent Learning All You Need in the StarCraft Multi-Agent Challenge? <https://doi.org/10.48550/ARXIV.2011.09533> Version Number: 1.
- [14] Ryan Donnelly. 2022. Optimal execution: A review. *Applied Mathematical Finance* 29, 3 (2022), 181–212.
- [15] Ryan Donnelly and Matthew Lorig. 2020. Optimal trading with differing trade signals. *Applied Mathematical Finance* 27, 4 (2020), 317–344.
- [16] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. 2021. Rigging the Lottery: Making All Tickets Winners. <https://doi.org/10.48550/arXiv.1911.11134> arXiv:1911.11134 [cs, stat].
- [17] Jin Fang, Jiacheng Weng, Yi Xiang, and Xinwen Zhang. 2022. Imitate then Transcend: Multi-Agent Optimal Execution with Dual-Window Denoise PPO. <https://doi.org/10.48550/arXiv.2206.10736> arXiv:2206.10736 [cs, q-fin].
- [18] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.* 23, 1, Article 120 (jan 2022), 39 pages.
- [19] Martin Forde, Leandro Sánchez-Betancourt, and Benjamin Smith. 2022. Optimal trade execution for Gaussian signals with power-law resilience. *Quantitative Finance* 22, 3 (2022), 585–596.
- [20] Sascha Frey, Kang Li, Peer Nagy, Silvia Sapor, Chris Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. 2023. JAX-LOB: A GPU-Accelerated limit order book simulator to unlock large scale reinforcement learning for trading. <https://doi.org/10.48550/arXiv.2308.13289> arXiv:2308.13289 [cs, q-fin].
- [21] Sascha Yves Frey, Kang Li, Peer Nagy, Silvia Sapor, Christopher Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. 2023. JAX-LOB: A GPU-Accelerated limit order book simulator to unlock large scale reinforcement learning for trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 583–591.
- [22] Trevor Gale, Erich Elsen, and Sara Hooker. 2019. The State of Sparsity in Deep Neural Networks. <https://doi.org/10.48550/arXiv.1902.09574> arXiv:1902.09574 [cs, stat].
- [23] Olivier Guéant. 2016. *The Financial Mathematics of Market Liquidity: From optimal execution to market making*. Vol. 33. CRC Press.
- [24] Ulrich Horst and Xiaoyu Xia. 2019. Multi-dimensional optimal trade execution under stochastic resilience. *Finance and Stochastics* 23 (2019), 889–923.
- [25] Ruihong Huang and Tomas Polak. 2011. LOBSTER: Limit Order Book Reconstruction System. <https://doi.org/10.2139/ssrn.1977207>
- [26] Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. 2023. Mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 619–627.
- [27] Qi Jin, Mihai Cucuringu, and Álvaro Cartea. 2023. Correlation Matrix Clustering for Statistical Arbitrage Portfolios. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAF '23)*. 557–564.
- [28] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=qrw7eXHTmYb>
- [29] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. 2021. BASE Layers: Simplifying Training of Large, Sparse Models. arXiv:2103.16716 [cs.CL] <https://arxiv.org/abs/2103.16716>
- [30] Diego León, Arbey Aragón, Javier Sandoval, Germán Hernández, Andrés Arévalo, and Jaime Niño. 2017. Clustering algorithms for Risk-Adjusted Portfolio Construction. *Procedia Computer Science* 108 (2017), 1334–1343. <https://doi.org/10.1016/j.procs.2017.05.185> International Conference on Computational Science, ICCS 2017, 12–14 June 2017, Zurich, Switzerland.
- [31] Chris Lu, Jakub Grudzien Kuba, Alistair Letcher, Luke Metz, Christian Schroeder de Witt, and Jakob Foerster. 2022. Discovered Policy Optimisation. <https://arxiv.org/abs/2203.02977>

- <https://doi.org/10.48550/arXiv.2210.05639> arXiv:2210.05639 [cs].
- [32] Xiao Ma, Shen-Yi Zhao, and Wu-Jun Li. 2019. Clustered Reinforcement Learning. arXiv:1906.02457 [cs.LG] <https://arxiv.org/abs/1906.02457>
- [33] Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Jackson, Samuel Coward, and Jakob Foerster. 2024. Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning. <https://doi.org/10.48550/arXiv.2402.16801> arXiv:2402.16801 [cs].
- [34] Peer Nagy, Jan-Peter Callies, and Stefan Zohren. 2023. Asynchronous Deep Double Duelling Q-Learning for Trading-Signal Execution in Limit Order Book Markets. *Frontiers in Artificial Intelligence* 6 (Sept. 2023), 1151003. <https://doi.org/10.3389/frai.2023.1151003> arXiv:2301.08688 [cs, q-fin].
- [35] Eyal Neuman, Wolfgang Stockinger, and Yufei Zhang. 2023. An offline learning approach to propagator models. *arXiv preprint arXiv:2309.02994* (2023).
- [36] Eyal Neuman and Moritz Voß. 2022. Optimal signal-adaptive trading with temporary and transient price impact. *SIAM Journal on Financial Mathematics* 13, 2 (2022), 551–575.
- [37] Brian Ning, Franco Ho Ting Lin, and Sebastian Jaimungal. 2021. Double deep Q-learning for optimal execution. *Applied Mathematical Finance* 28, 4 (2021), 361–380.
- [38] Johan Obando-Ceron, Ghada Sokar, Timon Willi, Clare Lyle, Jesse Farebrother, Jakob Foerster, Gintare Karolina Dziugaite, Doina Precup, and Pablo Samuel Castro. 2024. Mixtures of Experts Unlock Parameter Scaling for Deep RL. <http://arxiv.org/abs/2402.08609> arXiv:2402.08609 [cs].
- [39] Johan Obando-Ceron, Ghada Sokar, Timon Willi, Clare Lyle, Jesse Farebrother, Jakob Foerster, Gintare Karolina Dziugaite, Doina Precup, and Pablo Samuel Castro. 2024. Mixtures of Experts Unlock Parameter Scaling for Deep RL. <http://arxiv.org/abs/2402.08609> arXiv:2402.08609 [cs].
- [40] Johan Obando-Ceron, Ghada Sokar, Timon Willi, Clare Lyle, Jesse Farebrother, Jakob Foerster, Gintare Karolina Dziugaite, Doina Precup, and Pablo Samuel Castro. 2024. Mixtures of Experts Unlock Parameter Scaling for Deep RL. <http://arxiv.org/abs/2402.08609> arXiv:2402.08609 [cs].
- [41] Jochen Papenbrock and Peter Schwendner. 2015. Handling risk-on/risk-off dynamics with correlation regimes and correlation networks. *Financial Markets and Portfolio Management* 29, 2 (2015), 125–147.
- [42] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. 2024. From Sparse to Soft Mixtures of Experts. arXiv:2308.00951 [cs.LG] <https://arxiv.org/abs/2308.00951>
- [43] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. 2024. From Sparse to Soft Mixtures of Experts. <https://doi.org/10.48550/arXiv.2308.00951> arXiv:2308.00951 [cs].
- [44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. (2017). arXiv:1707.06347 arXiv preprint, available at arXiv:1707.06347.
- [45] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=B1ckMDqlg>
- [46] Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. 2019. Residual Policy Learning. <https://doi.org/10.48550/arXiv.1812.06298> arXiv:1812.06298 [cs].
- [47] Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, Wen-tau Yih, Jason Weston, and Xian Li. 2024. Branch-Train-MiX: Mixing Expert LLMs into a Mixture-of-Experts LLM. <https://arxiv.org/abs/2403.07816v1>
- [48] Vincenzo Tola, Fabrizio Lillo, Mauro Gallegati, and Rosario N. Mantegna. 2008. Cluster analysis for portfolio optimization. *Journal of Economic Dynamics and Control* 32, 1 (2008), 235–258. <https://doi.org/10.1016/j.jedc.2007.01.034> Applications of statistical physics in economics and finance.
- [49] U. von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and Computing* 17 (2007), 395–416. Issue 4.
- [50] Timon Willi, Akbir Khan, Newton Kwan, Mikayel Samvelyan, Chris Lu, and Jakob Foerster. 2023. Pax: Scalable Opponent Shaping in JAX. <https://github.com/ucl-dark/pax>
- [51] Timon Willi, Johan Obando-Ceron, Jakob Foerster, Karolina Dziugaite, and Pablo Samuel Castro. 2024. Mixture of Experts in a Mixture of RL settings. *arXiv preprint arXiv:2406.18420* (2024).
- [52] Zheng Xiong, Jacob Beck, and Shimon Whiteson. 2023. Universal Morphology Control via Contextual Modulation. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 38286–38300. <https://proceedings.mlr.press/v202/xiong23a.html> ISSN: 2640-3498.
- [53] Chuheng Zhang, Yitong Duan, Xiaoyu Chen, Jianyu Chen, Jian Li, and Li Zhao. 2023. Towards Generalizable Reinforcement Learning for Trade Execution. <https://doi.org/10.48550/arXiv.2307.11685> arXiv:2307.11685 [cs, q-fin, stat].
- [54] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. ST-MoE: Designing Stable and Transferable Sparse Expert Models. arXiv:2202.08906 [cs.CL] <https://arxiv.org/abs/2202.08906>

A Prior Knowledge in RL Agents

There are two ways we proposed in our research to give prior knowledge to the RL agents in the optimal execution task. One way is through clustering methods to feed the MoE with predetermined datasets. Another is through residual policy learning; recall that we employed the TWAP strategy as the baseline policy from which the RL agent learns to deviate. Below, we describe this in more detail.

A.1 Residual Policy Learning

In order to leverage prior knowledge, we incorporate residual policy learning (RPL) [46] into our optimal execution framework. This approach allows us to build upon well-established policies using deep reinforcement learning techniques. The core concept involves augmenting initial policies — in our case the time-weighted average price (TWAP) strategy — by learning residual functions that refine the initial policy.

Given an initial policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ mapping states $s \in \mathcal{S}$ to actions $a \in \mathcal{A} \subseteq \mathbb{R}^d$, we introduce a learned residual function $f_\theta(s) : \mathcal{S} \rightarrow \mathcal{A}$. This results in a residual policy $\pi_\theta : \mathcal{S} \rightarrow \mathcal{A}$ defined as:

$$\pi_\theta(s) = \pi(s) + f_\theta(s). \quad (7)$$

A key insight from [46] is that the gradient of the policy with respect to the parameters θ is independent of the initial policy π . This property allows us to employ policy gradient methods to optimize π_θ , even when dealing with non-differentiable initial policies.

In our specific implementation, $f_\theta(s)$ represents the TWAP strategy, while $\pi(s)$ captures the learned deviations from these baseline actions. This approach enables us to refine and adapt the TWAP strategies based on the complex dynamics of the market environment.